



Atividade prática Computação Bioinspirada

Você irá escolher um dos problemas abaixo, implementá-lo e analisá-lo em um Google Colab notebook. Cada problema é apresentado como um desafio real em uma empresa fictícia, detalhando a modelagem e sugerindo algoritmos bioinspirados. A entrega será através de um formulário (google forms) onde será solicitado um link (google colab) para o seu projeto.

Os trabalhos serão realizados em grupos de no máximo 3 integrantes. O formulário para submissão dos trabalhos e nomes dos integrantes de cada grupo será enviado através do grupo de whatsapp dos alunos da pós graduação.

Critérios de Avaliação (10 pontos)

A tabela abaixo apresenta os critérios e pesos que serão utilizados para avaliação dos trabalhos desenvolvidos.

- Descrição clara do cenário e modelagem: 2 pontos
- Implementação correta do algoritmo escolhido: 3 pontos
- Qualidade e legibilidade do código (comentários, modularização): 2 pontos
- Experimentos e visualização de resultados: 3 pontos

Problemas Sugeridos

Escalonamento de Tarefas com Restrições

Caso “GabioTech Task Masters”

A GabioTech é uma startup de automação industrial que fabrica peças personalizadas.

- Cenário real: milhares de jobs (tarefas) precisam passar por máquinas em sequência definida (ex.: jateamento → corte → polimento). Cada tarefa tem duração e dependências (não dá pra cortar antes de jatear). O objetivo é minimizar o makespan (tempo total até todas as peças ficarem prontas).

Modelagem do Problema

1. Variáveis:
 - Tarefas $T = \{t_1, \dots, t_n\}$
 - Duração d_i de cada t_i
 - Conjunto de dependências $\{(t_i \rightarrow t_j)\}$
2. Solução: uma permutação válida de Tarefas (ordem de execução).
3. Função-objetivo: tempo de conclusão total, penalizando fortemente ordens que violem dependências.
4. Número máximo de trabalhadores ou máquinas (tarefas em paralelo)

Dicas de Implementação

- Representação: cromossomo ou vetor de permutação.
- Algoritmos sugeridos:
 - Algoritmos Genéticos (AG) com crossover de permutação (order-crossover) e mutação swap.
 - Simulated Annealing (SA), realizando vizinhança por troca de duas tarefas.



- Ant Colony Optimization (ACO), onde formigas constroem sequências respeitando precedências.
- Validação: use ordenação topológica para verificar e penalizar violações.
- Visualização: Gantt chart simples (matplotlib) para comparar seqüências distintas.

Otimização do Layout de Antenas Wi-Fi

Caso “lallenNet WiSense”

A lallenNet é uma provedora de internet residencial que quer otimizar a cobertura Wi-Fi em prédios de condomínio.

- Cenário real: posicionar um número fixo de roteadores no mapa do prédio (planta em planta baixa) para maximizar a área de cobertura, evitando interferência por sobreposição excessiva.

Modelagem do Problema

1. Variáveis:
 - Número de antenas mm.
 - Coordenadas contínuas (x_k, y_k) dentro dos limites do prédio.
2. Função-objetivo:
 - Soma da cobertura (pontos cobertos) menos penalidade proporcional à área de sobreposição de sinais.
3. Dominio: $(x_k, y_k) \in \text{região poligonal (planta)}$.

Dicas de Implementação

- Representação: posição contínua para cada antena.
- Algoritmos sugeridos:
 - Particle Swarm Optimization (PSO), com partículas explorando o espaço contínuo.
 - AG com codificação real e crossover blend.
 - SA, mutando posições de antenas aleatoriamente.
- Cobertura: modelo simples de decaimento $S(d) = S_0 / (1 + d^2)$ ou disco de raio rr.
- Avaliação: discretize a planta em grade e conte células cobertas vs. sobrepostas.

Problema da Mochila com Múltiplos Constrangimentos

Caso “SantosPack Logistics”

A SantosPack é uma empresa de courier express que distribui caixas e encomendas.

- Cenário real: cada veículo de entrega tem limite de peso e volume. Selecionar quais encomendas carregar para maximizar o lucro (valor de entrega), sem exceder ambos os limites.

Modelagem do Problema

1. Itens: encomendas com peso w_i , volume v_i e valor p_i .
2. Capacidades: peso máximo W e volume máximo V .
3. Solução: vetor binário $x_i \in \{0, 1\}$ indicando inclusão.
4. Função-objetivo: $\sum p_i x_i$, penalizando se $\sum w_i x_i > W$ ou $\sum v_i x_i > V$.



Dicas de Implementação

- Representação: vetor binário de tamanho nn.
 - Algoritmos sugeridos:
 - AG (crossover one-point e mutação bit-flip).
 - PSO binário, adaptando velocidades a probabilidades de 0/1.
 - SA, flipando bits e aceitando penalizações progressivas.
 - Penalidades: adicionar valor negativo proporcional ao excesso de peso/volume.
 - Análise: experimente diferentes cargas e compare heurísticas em termos de valor vs. penalidade.
-

Estrutura Sugerida do Notebook (Google Colab)

1. Capa e Instruções
 - Título, nome(s) do(s) autor(es), problema escolhido, data.
 - Breve instrução de execução de todas as células.
2. Introdução
 - Descrição da empresa fictícia e cenário real.
 - Formulação matemática do problema.
3. Importação de Bibliotecas e Dados
 - numpy, matplotlib, pandas, bibliotecas de meta-heurísticas (ex.: DEAP).
 - Se necessário, carregar planta do prédio, lista de tarefas ou itens.
4. Modelagem e Função-Objetivo
 - Código que calcula custo, makespan ou cobertura para uma solução dada.
5. Implementação do Algoritmo Bioinspirado
 - Classe/funções principais do AG, PSO, SA ou ACO.
 - Parâmetros escolhidos (população, iterações, taxas).
6. Experimentos e Resultados
 - Rodar múltiplas seed = ... para estatística.
 - Gráficos de convergência (valor vs. iteração).
 - Visualizações (Gantt, planta com cobertura, gráfico valor vs. peso).
7. Análise e Discussão
 - Pontos fortes, limitações e possíveis melhorias.
8. Conclusão
 - Principais aprendizados e recomendações.